SALEM–SPENCER SETS AS A CROSS-PARADIGM SOLVER BENCHMARK: A WITNESS-SPLIT CAMPAIGN ON $\mathrm{r}_3(212)$ AND THE COLLAPSE OF AN LP-RESISTANT HARD POCKET

MEHMET ERGEZER

Abstract. Deciding whether a 44-element subset of $[1, 212]$ can avoid all three-term arithmetic progressions is the current frontier instance of the Salem–Spencer sequence $\mathrm{r}_3(N)$ (OEIS A003002). We present this family of finite combinatorial feasibility problems as a constraint-solving benchmark that separates solver technologies at three measured heights. No paradigm touches the monolithic instance: CP-SAT, HiGHS MIP, and CDCL all return unresolved after 24 hours, with the MIP dual bound pinned at 0.0. Under a witness-informed cube-and-conquer split with window-cardinality implied constraints — the single most effective pruning layer, worth 28 percentage points of unresolved rate — millions of subproblems close with zero feasible 44-sets, supporting but not proving the conjectured $\mathrm{r}_3(212) = 43$. Cross-paradigm analysis of the resistant residual then stratifies cleanly: an eight-hour HiGHS audit closes 25 of 45 survivors while leaving 20 LP-flat; CDCL (CaDiCaL) closes 18 of those 20, each with an independently verified DRAT proof; and the final two subproblems (T1c), which defeat CaDiCaL at 12 hours, fall to kissat 4.0.4 well within that budget, with DRAT certificates independently verified end-to-end against sha256-pinned formula files — refuting our own conjecture that the pocket was paradigm-resistant. A kissat survey then closes 99 of 100 sampled broad-residual subproblems within two hours, pricing the full certified sweep — and with it the unit-gap problem $\mathrm{r}_3(212) \in \{43, 44\}$ — as a bounded computation rather than an open search question. We release the tiered instances, verified certificates, deterministic generator, and a Lean encoding of T1c.

1. Introduction and background

Let $r_3(N)$ denote the maximum size of a subset of $[1, N]$ containing no nontrivial three-term arithmetic progression. Equivalently,

$$r_3(N) \;=\; \max\{\, |A| \;:\; A \subseteq [1, N],\ \text{no } a < b < c \text{ in } A \text{ satisfy } a + c = 2b \,\}.$$

The asymptotic study of $r_3(N)$ begins with Roth's theorem [Rot53] and continues through Salem–Spencer and Behrend-type lower-bound constructions [SS42, Beh46] and modern density-increment upper bounds, including work of Bloom–Sisask and Kelley–Meka [BS20, KM23]. Those results describe the large-N behavior of progression-free sets. The present paper is about a different but complementary problem: exact finite computation at the OEIS A003002 frontier.

OEIS A003002 tabulates exact values of $r_3(N)$. Cariboni's b-file currently reaches $N = 211$, where $r_3(211) = 43$ [OEI24, Car24]. The next natural decision problem is therefore:

Does there exist a 44-element 3-AP-free subset of $[1, 212]$?

Date: July 3, 2026.

2020 *Mathematics Subject Classification.* Primary 11B25; Secondary 11Y16, 68W30, 68V20, 90C10 .

Key words and phrases. constraint programming, benchmark instances, Salem–Spencer numbers, arithmetic progressions, OEIS A003002, CP-SAT, MIP, CDCL, DRAT proofs, formal verification, Lean .

ORCID: 0000-0001-6627-3667.

We found and independently verified a 43-element 3-AP-free subset of $[1, 212]$, so the lower bound $r_3(212) \geq 43$ is settled. The upper-bound question is whether $r_3(212) \leq 43$. Since $r_3(211) = 43$, any hypothetical 44-set in $[1, 212]$ must contain both endpoints 1 and 212; otherwise it would translate or restrict to a 44-set in $[1, 211]$. Thus the target is a single finite feasibility problem with a strong necessary endpoint condition.

Finite feasibility problems of this Ramsey-combinatorial flavor have been a showcase for modern SAT and constraint solving: the Boolean Pythagorean triples problem [HKM16], Schur number five [Heu18], and Keller's conjecture in dimension seven [BHMN22] were each closed by massively parallel CDCL search with machine-checkable proofs. The Salem–Spencer family sits in the same problem class — Boolean variables, one short linear constraint per arithmetic progression, a cardinality target — and, as this paper documents, it separates solver technologies unusually sharply. No paradigm touches the monolithic frontier instance in 24 hours. Under a witness-informed split, LP-paradigm methods (CP-SAT propagation and LP-relaxation-based MIP) fail uniformly on a residual pocket whose LP dual bound stays pinned at zero, while CDCL closes it — but only implementation-dependently: the two hardest audited subproblems defeat CaDiCaL at a 12-hour wall yet fall to kissat well within that budget, with independently verified DRAT certificates. That layered separation is precisely what makes the family interesting as a benchmark: it is natural, scalable in N, has verifiable ground truth at solved sizes, and grades solvers along a measured difficulty ladder rather than a single pass/fail line.

We attacked this feasibility problem computationally using a reproducible CP-SAT architecture. The model uses Boolean variables x_i, one linear constraint for each 3-term arithmetic progression, the decision equality $\sum_i x_i = 44$, endpoint forcing, reflection symmetry breaking, and window-cardinality inequalities derived from the known values of $r_3(L)$ for $L \leq 211$. To make the search tractable, we split the problem by assigning high-degree variables from a verified 43-witness, generating a deterministic depth-24 chunk space, and then refine residual `UNKNOWN` chunks recursively.

The campaign did not produce a formal proof of $r_3(212) = 43$. It did produce four concrete outcomes.

First, we observed zero feasible 44-sets across millions of CP-SAT subproblems, including broad chunks, recursive refinements, wall-cap recalibrations, and targeted A/B experiments. This is empirical evidence for the expected value $r_3(212) = 43$, but it is not a certificate.

Second, we found that OEIS window-cardinality inequalities are essential. On controlled broad-pass ranges, adding these inequalities reduced the `UNKNOWN` rate by roughly 28 percentage points and also reduced aggregate solver time. They are the main successful pruning layer of the campaign.

Third, we identified and stratified a structural hard pocket. The largest retained broad run over 100,000 depth-24 chunks left 6,071 `UNKNOWN` chunks. A uniform random 100-chunk sample of those 6,071 was passed through a 300-second recap, leaving 45 resistant survivors. Re-attacking those 45 chunks with HiGHS, an LP-relaxation-based MIP solver, closed 0/45 in one-hour-per-chunk runs. A full eight-hour audit later closed 25/45, but left 20/45 `UNKNOWN`, all with dual bound still pinned at 0.0. A subsequent pure CDCL/SAT attack (CaDiCaL via PySAT) closed 18/20 of that LP-flat subset, while two chunks — T1c — remained `UNKNOWN` even under a 12-hour pure-CDCL cap and a 4-hour windowed-CDCL diagnostic.

Fourth, the pocket collapses under a current competition-grade CDCL solver. Kissat 4.0.4 refutes both T1c chunks on the pure 3-AP/cardinality encoding, well within the budget under which CaDiCaL failed, and both refutations are certified end-to-end: `drat-trim` independently verified the DRAT proofs against sha256-pinned formula files (§4.4). A kissat survey over a fresh uniform 100-chunk sample of the broad residual closes 99/100 within a 2-hour cap. The obstruction that survived CP-SAT tuning, HiGHS, and CaDiCaL is therefore not a paradigm boundary but a solver-implementation boundary — a finding we could only make, and can only state precisely, because the pipeline isolates and archives the instances on which implementations diverge.

This paper's contribution is the benchmark and the cross-paradigm empirical study, not a new exact value of A003002. We provide:

1. A tiered benchmark release spanning a measured difficulty ladder: the 25 HiGHS-closable T1a chunks, the 18 CaDiCaL-closable T1b-minus-T1c chunks, the 2 T1c chunks that separate CaDiCaL from kissat, the 6,071 broad `UNKNOWN` chunks from the 100k expansion batch, and a deterministic generator for the full depth-24 sweep.
2. A characterization of the structural hard pocket across three solver paradigms and two CDCL implementations, including the refutation of our own working conjecture: the pocket that appeared paradigm-invariant is closed outright by kissat 4.0.4, so the boundary was implementation, not paradigm.
3. A reproducible witness-split plus window-cardinality architecture for exact $r_3(N)$ upper-bound search, parametric in (N, K) and reusable at adjacent frontier sizes. The problem-specific window-cardinality inequalities are, empirically, worth more than any generic solver lever we tested.
4. A detailed empirical account of the $N = 212$, $K = 44$ campaign, including broad-pass counts, refinement behavior, solver-time tradeoffs, and the controlled A/B experiments behind each design decision.

For reference, the benchmark tier notation used throughout the paper is:

Symbol	Meaning
T1	the 45 chunks that survived the 300-s recap of a random 100-chunk sample from the 6,071 broad UNKNOWNs
T1a	the 25 T1 chunks closed by the full 8-h HiGHS audit
T1b	the 20 T1 chunks left UNKNOWN by the full 8-h HiGHS audit, all with dual bound 0.0
T1b $\setminus$ T1c	the 18 T1b chunks closed by CDCL and independently verified by DRAT checking
T1c	the final two chunks $\{40959, 48895\}$: resist CP-SAT, HiGHS, and CaDiCaL; refuted by kissat 4.0.4 with verified DRAT proofs
T2	the 6,071 UNKNOWN chunks from the 100k broad expansion batch
T3	the full 12,582,912-chunk AP-pruned depth-24 sweep

1.1. **Related work. Split-and-solve search.** The witness-split architecture of §2.2 belongs to the cube-and-conquer family [HKWB12]: partition the search space by fixing a prefix of variable assignments (cubes), then solve each cube with a complete solver. The differences are the cube-selection policy and the refinement loop. Classical cube-and-conquer selects split variables by lookahead heuristics; we select them by AP-incidence degree *within a verified extremal witness*, exploiting the problem-specific intuition that a hypothetical 44-set must interact heavily with the densest regions of the known 43-set. Timed-out cubes are then re-split recursively (§2.4), which parallels iterative cube deepening.

Implied constraints and streamliners. The window-cardinality inequalities of §2.3 are implied constraints in the standard CP-modeling sense: valid inequalities not inferable by the solver from the base model. They are derived from previously *computed* values of the same sequence — the solved prefix of A003002 prunes the frontier instance. This is soundness-preserving, in contrast to streamlined constraint reasoning [GS04], which deliberately sacrifices completeness; it is closer in spirit to bound propagation from tabulated subproblem optima. Empirically these inequalities dominate every generic lever we tested (§3.2, §4.4).

SAT and CP in combinatorial mathematics. Beyond the celebrated results cited above, exact computation of Salem–Spencer values has its own lineage: Dybizbański [Dyb12] computed $r_3(N)$ through $N = 123$ with a tailored branch-and-bound, and Cariboni's OEIS b-file [Car24] extends the sequence to $N = 211$. Our campaign differs in attacking the frontier with general-purpose solvers plus problem-specific pruning, and in reporting the negative space — which subproblems resist and why — rather than only the closed values.

Benchmarks and certified solving. Curated instance families — CSPLib [GW99], MIPLIB [GHG+21], the SAT Competition suites — have repeatedly steered solver research toward documented failure modes. The tiered release of §5 is offered in that tradition: a natural family, scalable in N, with verifiable ground truth at solved sizes and a measured cross-paradigm difficulty ladder. On the certification side, the CDCL-closed tier ships with DRAT proofs verified by `drat-trim` [HHB13, CFHH+17]; the absence of comparable certificates for the CP-SAT and MIP closures (§6.2) is an instance of the gap that auditable constraint solving [GMN22] aims to close.

1.2. **Organization.** The rest of the paper is organized as follows. §2 describes the architecture. §3 reports the computational campaign, beginning with the monolithic baseline. §4 analyzes the hard pocket across HiGHS, CaDiCaL, and kissat, ending in its collapse and the refutation of our working conjecture. §5 releases the tiered benchmark and prices the remaining path to a full certificate. §6 discusses transferability, limitations, and what the campaign suggests about finite r_3 upper-bound search.

2. Architecture

The campaign rests on five components: a decision-form CP-SAT model of the 3-AP-free subset problem (§2.1), a witness-variable splitting scheme that produces tractable subproblems (§2.2), a window-cardinality pruning family derived from OEIS A003002 (§2.3), a recursive refinement loop for residual `UNKNOWN` chunks (§2.4), and the SLURM-side engineering that makes the workload reproducible (§2.5). Each component is independent of the others — any could be replaced by a stronger variant without disturbing the rest — and together they define the proof attempt on $r_3(212) \leq 43$.

2.1. **Decision-form CP-SAT model.** Fix $N \geq 3$ and $K \geq 3$. The standard CP-SAT encoding of "does there exist a 3-AP-free subset $A \subseteq [1, N]$ with `|A| ≥ K`?" introduces Boolean variables `x_i ∈ \{0, 1\}` for $i \in [1, N]$ with $x_i = 1$ iff $i \in A$, and adds the constraints

```
x_a + x_b + x_c ≤ 2     for every 3-AP triple (a, b, c),
sum_i x_i ≥ K.
```

We adopt the **decision-form** variant in which the second inequality is replaced by the equality

```
sum_i x_i = K.
```

The decision-form encoding is tighter for propagation, not for logical interpretation of `UNKNOWN`: an `UNKNOWN` return on either encoding is still consistent with both feasibility and infeasibility. The practical advantage of $\sum_i x_i = K$ is that CP-SAT and pseudo-Boolean propagators see both the lower and upper cardinality directions. Branches that would require more than K selected values, or too few remaining unfixed values to reach K, can be cut earlier than in the inequality form. For our purposes (proving the upper bound $r_3(N) \leq K - 1$), the equality form is therefore the right primitive.

For $N = 212$, $K = 44$, the model has 212 Boolean variables and 11,130 3-AP triple inequalities. We add the lex symmetry-breaking constraint

```
(x_1, x_2, ..., x_N)  ≥_lex  (x_N, x_{N-1}, ..., x_1)
```

to factor out the reflection $i \mapsto N + 1 - i$. The opposite orientation would be equivalent; the implementation uses `≥_lex`. We do not add other symmetries. After endpoint forcing, reflection is the only order-preserving symmetry we exploit.

We also apply **endpoint forcing** specific to the $r_3(212)$ case. Since OEIS A003002 records $r_3(211) = 43$, any 44-element 3-AP-free subset of $[1, 212]$ must contain both endpoints 1 and 212: if 1 were absent, the set would be a 44-element 3-AP-free subset of $[2, 212]$, which shifts to $[1, 211]$; if 212 were absent, it would already lie in $[1, 211]$. Either case contradicts $r_3(211) = 43$. We add $x_1 = x_{\{}212\} = 1$ as ground assumptions in every chunk.

Branching: variable selection by AP-incidence degree (the number of 3-AP triples a variable appears in), value selection `min`, and `fixed_search` to disable CP-SAT's portfolio search. The first two target the most-constrained variables first; the third reduces solver-policy variation across runs, which is necessary for the A/B experiments of §3.2 and §3.5.

2.2. **Witness-variable splitting.** The decision-form model on the full range $[1, 212]$ is already at the limit of single-call CP-SAT solving in reasonable wall time. To break it into tractable subproblems we use a cube-and-conquer-style split [HKWB12] in which the cubes are chosen not by lookahead but from the verified 43-element lower-bound witness `A_43 ⊂ [1, 212]` (§2.1, §3.1). The splitting proceeds in three steps.

Degree ranking. For each $v \in A_43$, compute the AP-incidence degree `deg(v) = |\{(a, b, c) : v \in \{a, b, c\}, b - a = c - b\}|`. Sort `A_43` by `deg(v)` in decreasing order. Pick the top $d = 24$ witness values; we refer to this prefix as the **broad split prefix**.

Combinatorial enumeration. For each of the `2^24 = 16,777,216` IN/OUT assignments of the broad split prefix, instantiate a **chunk**: the decision-form model with x_v fixed to the chosen value for every `v` in the prefix. A chunk is identified by its 24-bit chunk ID.

AP-prefix pruning. Many chunks are immediately infeasible at the 3-AP level alone: if the broad split prefix forces three pinned-IN values that form a 3-AP, the chunk has no feasible completion and need not be sent to CP-SAT. We perform this check during chunk emission and skip such chunks. The surviving chunk count for $N = 212$, $K = 44$ at depth 24 is 12,582,912, roughly 75% of the raw `2^24` count.

The choice $d = 24$ is empirical: larger d produces more chunks but each is easier; smaller d produces fewer chunks but each is harder. At $d = 24$ the per-chunk solver time at the 60-s wall cap has a heavy tail but a tractable median (the broad pass closes `~94%` of expansion chunks within 60 s — see §3.1), which makes the depth 24 choice a workable broad layer.

2.3. **Window-cardinality pruning from OEIS A003002.** The 3-AP-free subset problem admits a family of implied constraints — valid inequalities that are not derivable by the solver from the 3-AP triple inequalities alone. For any window $[a, a + L - 1] \subseteq [1, N]$ and any length L with $r_3(L) < L$,

```
sum_{i ∈ [a, a+L-1]}  x_i  ≤  r_3(L).
```

This is valid because `\{i ∈ A : i ∈ [a, a + L - 1]\}` is itself a 3-AP-free subset of an L-element interval, hence of size at most $r_3(L)$. Crucially, the right-hand side $r_3(L)$ is *constant* with respect to `a`, so the family is a single constant per window length, not a learned bound.

We populate the right-hand sides from the OEIS A003002 b-file (Cariboni's tabulation, valid for $L \leq 211$). For $N = 212$, $K = 44$ we generate window constraints for every available length L with $r_3(L) < min(L, K)$ and every `a ∈ \{1, 2, ..., 212 - L + 1\}`. The resulting model contains 22,154 window inequalities.

Empirically the window family is the single most impactful CP-SAT intervention in the campaign: the controlled A/Bs of §3.2 show a ~28-percentage-point reduction in `UNK` rate at the 60-s wall cap when window-bounds are enabled. We use it as the default configuration for every solver call beyond the calibration batch.

2.4. **Recursive refinement.** A chunk that returns `UNKNOWN` at the broad layer is refined by extending the broad split prefix with the next m witness values (by AP-incidence degree) and re-emitting the resulting `2^m` subchunks (less AP-prefix pruning) to the solver under a longer wall cap. The depths and per-step parameters used in the campaign are:

Level	Depth	Step size m	Per-chunk wall cap
Broad	24	—	60 s
L1	40	+16	60 s
L2 (tail)	48	+8	60 s
L3 (level3)	56	+8	60 s
L4 (level4)	64	+8	600 s

Each level is fed only the `UNKNOWN` residuals of the prior level, so the work at deeper levels is concentrated on the hard tail. The expected per-level fan-out `2^m` is mitigated by AP-prefix pruning, which becomes more aggressive at deeper levels (more pinned-IN values, more chances for a 3-AP among them). For example, the Sample-500 L1 stream emits 2,076,105 rows from `500 × 2^16` nominal descendants, a `~6%` survival rate after pruning.

Refinement is the campaign's primary tool for closing residual `UNKNOWN` chunks. It reliably closes individual deep chunks (§3.1 reports 6/6 closure at L4). It does not generalize cheaply to the full 6,071-chunk expansion residual because the per-chunk cost grows roughly geometrically with depth.

2.5. **SLURM engineering.** The campaign runs on the UMass Unity SLURM cluster. The workload pipeline is implemented as a small collection of Python scripts, each with a single responsibility:

- **`r3_slurm_emit.py`** — generates the chunk-ID list for a given `(N, K, split-vars, depth, range)` and emits a SLURM array `sbatch` script that fans out the chunks across array tasks. The `--chunks-per-task` flag controls the granularity of fan-out.
- **`r3_split_cpsat.py`** — the per-task driver. Reads its slice of chunk IDs, instantiates the CP-SAT model for each, runs the solver under the per-chunk wall cap, and appends one JSONL row per chunk to a shard file.
- **`r3_collect.py`** — merges shard files into the canonical batch output. Handles deduplication if a chunk was solved more than once (e.g., during retries).
- **`r3_tail_emit.py`** — emits the next-level refinement workload from the `UNKNOWN` rows of a parent batch. Supports multi-parent inputs (each input JSONL contributing its own parent prefix) and templated output naming via a parent-tag regex.
- **`r3_proof_manager.py`** — tracks the proof-state graph across levels, identifying which chunks have been closed at which depth and which remain open.
- **`r3_verify.py`** — independent triple-enumeration verifier for any candidate K-element witness.

Two engineering decisions are worth highlighting because they affect reproducibility:

Atomic shard renames. Each per-task driver writes its shard to a temporary `.tmp` path keyed by the SLURM job and array IDs, and renames it to `shard.jsonl` on successful completion. A killed or pre-empted task leaves only the temporary file, which is ignored by the collector. This pattern prevents partial output from corrupting downstream collection without requiring any locking.

Deterministic inputs. Every chunk ID maps to a fixed prefix assignment, and every CP-SAT call uses the same solver seed and fixed-search branching. This is necessary for the lever A/B experiments of §3.5: a "no measurable effect" claim is only defensible if the baseline arm is reproducible under the same software stack.

The end-to-end pipeline is driven by `unity_handoff.sh`, which runs the broad pass, the recap studies, the refinement levels, and the HiGHS attack as separately resumable phases. A complete campaign reproduction on a fresh allocation requires the OEIS b-file, the verified 43-witness, the Python environment (OR-Tools, `highspy`, PySAT/CaDiCaL, and `drat-trim`), and a SLURM allocation; everything else is generated by the scripts.

3. Empirical campaign

This section reports the workload, results, and key A/B tests of the $r_3(212) \leq 43$ campaign. All experiments were run on the UMass Unity SLURM cluster against the architecture described in §2. The headline numbers: millions of CP-SAT subproblems solved, 0 `FEASIBLE` rows, and a 6.07% `UNKNOWN` residual in the largest retained broad batch that motivates the structural analysis in §4.

3.1. Monolithic baseline.

Before any splitting, we hand the full unsplit decision instance — 212 Boolean variables, 11,130 3-AP inequalities, $\sum_i x_i = 44$, endpoint forcing, with and without the 22,154 window-cardinality inequalities — to each solver paradigm with a 24-hour wall cap.

Paradigm	Windows	Wall	Status	Progress at cap
CP-SAT (8 workers)	on	24 h	`UNKNOWN`	50,127,252 conflicts
CP-SAT (8 workers)	off	24 h	`UNKNOWN`	814,459,413 conflicts
HiGHS MIP (8 threads)	on	24 h	`UNKNOWN`	837,301 nodes, dual 0.0
HiGHS MIP (8 threads)	off	24 h	`UNKNOWN`	6,800,276 nodes, dual 0.0
CDCL (1 core)	on	24 h	`UNKNOWN`	—
CDCL (1 core)	off	24 h	`UNKNOWN`	—

All six cells return `UNKNOWN` at the cap. The two HiGHS runs explore hundreds of thousands to millions of MIP nodes without lifting the dual bound off 0.0 — the same LP-flatness signature that later characterizes the hard pocket (§4.3). CP-SAT burns through roughly 8×10^8 conflicts on the bare encoding without a refutation. (Conflict counts for the two CDCL cells were not retained by the PySAT wrapper; both cells ran to the full cap without a result.) No paradigm resolves the monolithic instance, which is the empirical justification for the witness-split architecture of §2.2: the campaign's progress comes from the split and the window bounds, not from raw solver time on the whole problem. (The windowed CDCL cell required 64 GB at encode time; the window family is expensive in CNF form.)

3.2. Chunk-count breakdown.

The campaign proceeded in three layers of increasing scope, plus a recursive-refinement loop on the `UNKNOWN` residuals.

Verified lower bound. A 43-element 3-AP-free subset of $[1, 212]$ is recorded in the repository's witness JSON and re-verified by `r3_verify.py` against an independent triple-enumeration check.

This witness fixes the campaign's $K = 44$ decision-mode target and supplies the seed for the depth-24 broad split (§2.2).

Broad pass at depth 24, 60-s wall. The retained broad-pass logs include both pre-window and window-bound runs:

Range	Chunks	INFEASIBLE	UNKNOWN	UNK rate
Calibration, no window bounds, $[575, 1575)$	1,000	799	201	20.10%
Bounded, no window bounds, $[1575, 11575)$	10,000	6,967	3,033	30.33%
Bounded, with window bounds, $[1575, 11575)$	10,000	9,835	165	1.65%
Expansion, with window bounds, $[11575, 111575)$	100,000	93,929	6,071	6.07%

The 20.10% calibration rate was run *before* window-bounds were enabled and is included for reference only; once window-cardinality pruning is on (see §3.2), the residual sits in the 1–6% band depending on the chunk-ID range.

Recursive refinement of `UNKNOWN` chunks. Every chunk timing out at the broad layer can be refined by extending the witness-pin prefix with the next-degree split variables and re-solving the resulting subchunks. The number of emitted rows is not a raw `2^d` fan-out: AP-prefix pruning eliminates many descendants before a solver call is made. The retained refinement diagnostics are shown below. The depth labels follow the L1/L2/L3/L4 refinement ladder of §2.4.

Stream	Source	Depth added	Rows emitted	INFEASIBLE	UNKNOWN
Sample-100 L1	random sample of 100 broad UNKs	+16 (to depth 40)	299,375	299,374	1
Sample-100 tail	one Sample-100 residual	+8 (to depth 48)	8	8	0
Sample-500 L1	stratified sample of 500 broad UNKs	+16 (to depth 40)	2,076,105	2,076,095	10
Sample-500 tail8	10 Sample-500 residuals	+8 (to depth 48)	76	68	8
Sample-500 level3	8 tail8 residuals	+8 (to depth 56)	8	2	6
Sample-500 level4	6 level3 residuals at 600-s wall	+8 (to depth 64)	6	6	0

The final Sample-500 level4 cleanup closed all 6 deep residuals within the 600-s cap, with a worst-case solver time of 599.74 s — within 0.26 s of the cap, so this cleanup is at the practical limit of the refinement strategy at its current parameter settings. Crucially, none of these levels produced a `FEASIBLE` row.

The 6,071 `UNKNOWN` chunks of the expansion batch were *not* fully refined by this campaign. Three subsets of them were sampled for diagnostic experiments: a uniform random 100-chunk recap study (§3.2 and §4.2), a structural-mining analysis (§4.1), and the HiGHS attack of §4.3.

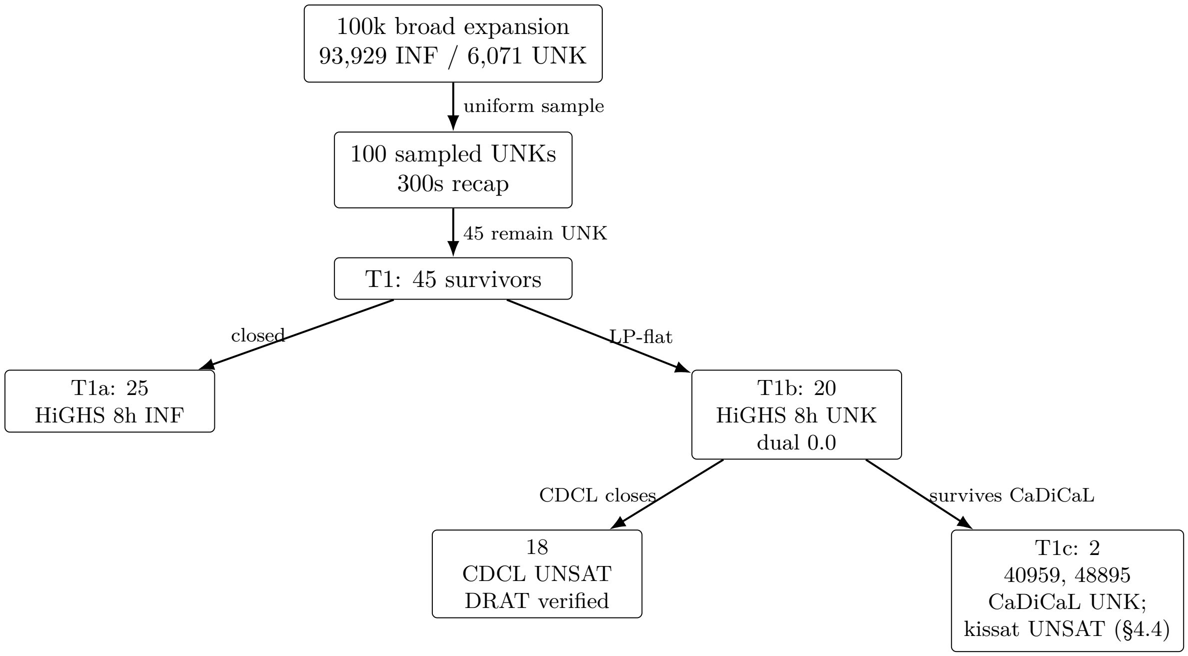

FIGURE 1. Funnel from the retained 100k broad expansion batch to the final two-chunk T1c residual. T1 is a 45-chunk subset produced by recapping a uniform random 100-chunk sample of the 6,071 broad UNKNOWN chunks at 300 seconds. T1c, which survives CaDiCaL, is later refuted by kissat 4.0.4 (§4.4).

3.3. **The window-cardinality A/B.** The single most impactful intervention in the campaign was adding window-cardinality inequalities derived from the OEIS A003002 b-file. For every window $[a, a+L-1] \subseteq [1, 212]$ and every length L with $r_3(L) < min(L, K)$, we add `sum_{i ∈ window} x_i ≤ r_3(L)`. For $N = 212$, $K = 44$ this generates 22,154 inequalities, on top of the 11,130 3-AP triple inequalities and the symmetry-breaking lex constraint.

We measured the effect on three batches of varying size:

Batch	Chunks	UNK without window-bounds	UNK with window-bounds	Reduction
$[575, 675)$	100	27.00%	0.00%	`-27.0 pp`
$[1575, 11575)$	10,000	30.33%	1.65%	`-28.7 pp`
$[11575, 111575)$	100,000	n/a	6.07%	—

Both controlled A/Bs show a roughly 28-percentage-point reduction in `UNK` rate at the 60-s broad wall cap. The window-bound family is strictly stronger than the baseline 3-AP family at this problem size. In the retained logs it also lowered total broad-pass solver time, because many formerly hard chunks closed early under the extra bounds.

The $[575, 675)$ A/B is a 100-chunk subrange of the calibration range reported in §3.1; the difference between its 27.00% no-window `UNK` rate and the calibration range's 20.10% reflects the structural non-uniformity of chunk-ID space rather than a change in model configuration.

However, the `UNK` rate is non-monotone in the chunk-ID range: the small controlled A/B on $[575, 675)$ closes to 0%, the 10k bounded batch sits at 1.65%, and the 100k expansion batch sits at 6.07%.

This indicates that the chunk-ID space is structurally non-uniform — later chunk IDs (which encode different witness-pin assignments) are harder in a way that window-bounds do not fully compensate for. The bucket analysis in §4.1 quantifies this non-uniformity.

3.4. **Solver configuration and engineering.** All CP-SAT calls use the following configuration unless explicitly overridden in an experiment:

- Model: decision-mode $\sum_i x_i = 44$, 11,130 3-AP linear inequalities, 22,154 window-cardinality inequalities, lex reflection symmetry break, endpoint forcing $x_1 = x_{\{}212\} = 1$.
- Branch strategy: variable selection by AP-incidence degree, value selection `min`, `fixed_search` to disable CP-SAT's portfolio.
- Solver: OR-Tools CP-SAT [PF24], 8 workers per task, fixed solver seed and fixed search parameters, wall cap as noted (most commonly 60-s for broad, 300-s and 600-s for recap experiments).

The SLURM emitter, shard collector, tail emitter, and proof-state manager implement the workload pipeline. Output is written as line-delimited JSON with one record per chunk; shards are written to a temporary path and atomically renamed on completion so that partial output from killed tasks cannot corrupt downstream collection. The full campaign is reproducible end-to-end via the `unity_handoff.sh` driver.

3.5. **Zero FEASIBLE across the campaign.** Aggregating across the monolithic baseline (§3.1), the broad pass, the refinement loop, the recap studies, the lever experiments of §3.6, the HiGHS attack of §4.3, the CDCL/SAT and proof-producing reruns of §4.3, and the kissat re-attack and T2 survey of §4.4, the campaign solved millions of `(N, K, fixed_in, fixed_out)` subproblems, including prefix-closure work not tabulated above. The number of subproblems returning `FEASIBLE` or `SAT` is **zero**.

The `FEASIBLE` count is the only signal that would directly disprove $r_3(212) \leq 43$; its absence does not constitute a proof of the upper bound, but it is strong empirical support, especially given that the campaign forced the endpoints 1 and 212, which any 44-set must contain by the known value $r_3(211) = 43$, and then explored a depth-24 prefix of the verified 43-witness in the processed chunk ranges. Within the processed expansion range, a 44-element 3-AP-free subset of $[1, 212]$, if one existed, would have to lie in the 6,071 unresolved broad `UNKNOWN` chunks (or in their deeper refinements). Globally, the much larger unprocessed remainder of the full depth-24 sweep remains open as well.

3.6. **Search-tuning levers and their effect sizes.** We tested five CP-SAT-side search-tuning levers. The full lever inventory, including the HiGHS substitution and the pair-AND Tseitin experiment, is consolidated in §4.5; here we report only the three CP-SAT-side levers that operate at the broad layer.

Lever	Mechanism	Bucket	Baseline UNK	Treatment UNK
Split-vars reorder	Permute the depth-24 prefix so hot pins (values `{68, 75, 70, 76, 91}`) lead	$[61575, 66575)$	13.60%	12.96%
Wall-cap extension 60s → 300s	Same model, same prefix, longer per-chunk budget	random 100 from expansion `UNK`	100.00%	45.00%
Recursive deepening	Refine `UNK` to depth 64 at 600-s cap	6 L3 residuals	100.00%	0.00%

The reorder lever was tested for completeness but is expected to be a null result on principle: the depth-24 broad split iterates over the `2^24` IN/OUT prefix assignments, and within-prefix variable ordering does not change the set of surviving chunks under AP-prefix pruning — only the order in which they are emitted to SLURM. The within-noise result confirms this.

The wall-cap extension lever moves the residual substantially on the first doubling but exhibits a saturating closure curve under further extension. The 60s → 120s → 300s series is analyzed in §4.2; in brief, the marginal close-rate per additional second of wall drops sharply past 300-s, which is why the campaign did not pursue 600-s or 1200-s broad-layer reruns on the full 6,071-chunk expansion residual.

The recursive-deepening lever, applied locally to small residual sets, reliably closes individual deep `UNK` chunks (6/6 at the L4 level in §3.1). But the levers it implies — picking the next-degree split variables and re-solving up to `2^8` descendant assignments at a longer wall — do not generalize cheaply to the full 6,071-chunk expansion residual. A naïve depth-32 rerun on every expansion `UNK` would emit roughly 1.5 million subproblems and require an order of magnitude more CPU time than the original broad pass, with no guarantee of closing the hard core analyzed in §4.

3.7. **Compute budget.** The dominant retained costs are the 100k broad expansion, the sample-500 refinement, and the full-T1 long-wall HiGHS audit; the CDCL and certificate diagnostics are comparatively cheap. The total retained-log estimate is roughly 7,850 worker-hours; Appendix A itemizes the accounting per campaign layer. A full depth-24 sweep of the 12,582,912 AP-pruned chunks of the witness-split lattice was outside this campaign's budget under the CP-SAT-first parameters used here; §5.3 reprices the sweep in light of the kissat results of §4.4, which shift the bottleneck from search to proof logging and verification at scale.

4. The structural hard pocket

The headline empirical finding of the campaign is not the absence of a 44-element 3-AP-free subset of $[1, 212]$ — which we expected from the OEIS A003002 extrapolation $r_3(212) = 43$ — but the existence, stratification, and eventual collapse of a small, robust subset of broad subproblems that resisted every solver we initially threw at them. We refer to this subset as the *hard pocket*. This section characterizes it empirically, summarizes the interventions in the order we tried them, and ends with the kissat re-attack that resolved it (§4.4) — a resolution that refutes our own working conjecture (§4.6) and relocates the hardness boundary from solver paradigm to solver implementation.

4.1. **Empirical signature on the broad population.** The 100,000-chunk window-bound broad batch over chunk-id range $[11575, 111575)$ produced 93,929 `INFEASIBLE` chunks, 6,071 `UNKNOWN` chunks, and 0 `FEASIBLE` chunks under a 60-second per-chunk wall cap. The 6.07% UNKNOWN rate is non-uniform in two distinct senses.

Bucket non-uniformity. Partitioned into twenty 5,000-chunk buckets, the UNKNOWN rate varies from 1.64% to 13.60%, with the worst bucket $[61575, 66575)$ at 13.60%. The longest contiguous UNKNOWN run found in this batch was length 27 at chunk IDs 98277..98303; many other runs have length around 15. These contiguous runs are a strong indication that UNKNOWN status is driven by joint structural properties of the depth-24 witness-pin assignment, not by stochastic solver behavior.

Pin-OUT enrichment. For each of the 24 broad-split witness variables we computed the conditional UNKNOWN rate given that variable's IN/OUT assignment in the chunk. Five variables — values 68, 75, 70, 76, 91, all in the dense middle cluster $[67, 91]$ of the verified 43-witness — show a strong asymmetry: their conditional UNKNOWN rate is roughly 2.4% when pinned IN and 9.7% when pinned OUT (Fisher log-odds +3.5). The signal is consistent across the five values. We refer to a chunk in which all five hot pins are forced OUT as a *middle-out chunk*. Middle-out chunks account for a disproportionate share of the UNKNOWN tail of the broad batch.

4.2. **The 300s-resistant tail does not collapse.** To test whether the broad-pass UNKNOWNs were merely "slow" rather than structurally hard, we ran two wall-cap recalibration experiments on a uniform random sample of 100 UNKNOWN chunks drawn from the 6,071 baseline. The results show a saturating, not exponential, closure curve:

Wall cap	INFEASIBLE	UNKNOWN	Close rate over baseline
60 s	0	100	0%
120 s	31	69	31%
300 s	55	45	55%

Going from 60s to 120s (2x) closed 31 chunks. Going from 120s to 300s (2.5x further) closed only 24 more. Naïve extrapolation to 600s predicts perhaps 10–15 additional closures. The hard tail does not vanish under more time.

We applied the same structural mining procedure (single-pin, pair, and triple log-odds enrichment relative to a matched `INFEASIBLE` sample) to the 45 `UNKNOWN` chunks that survived the 300s cap. The broad pin-OUT signature weakens substantially. The best high-coverage pair, $[91 = OUT, 48 = OUT]$, covers 66.67% of the survivors with log-odds +1.226 — a modest effect size, far from the +3.5 signal seen on the 6,071-row population. Top triples have stronger log-odds but each covers only a handful of cases. Hamming-distance clustering of survivor `fixed_in` sets shows multiple small clusters rather than a single dominant niche. We interpret this as evidence that the 300s-resistant subproblems share moderate pin-OUT structure but spread across many distinct sub-pockets of the depth-24 assignment lattice.

4.3. **LP-paradigm failure and the CDCL break.** The 300s-resistant pocket might in principle be a CP-SAT-specific phenomenon — an artifact of constraint propagation's inability to exploit implicit LP structure. To test this, we re-attacked all 45 survivors with the HiGHS open-source MIP solver [HH18], which combines branch-and-bound, LP relaxation, presolve, cutting planes, and primal heuristics. We used a one-hour wall cap per chunk, 8 threads per chunk, and the identical constraint set (decision-form $\sum x_i = 44$, all 11,130 3-AP triple inequalities, all 22,154 window-cardinality inequalities, plus the broad chunk's `fixed_in` and `fixed_out` assignments tightened into variable bounds).

The one-hour result was negative:

The two rows in the following table are not a common-sample comparison: the HiGHS attack targets precisely the 45 chunks that survived the 300-s CP-SAT recap, i.e. the harder subset of the 100 recap inputs.

Solver	Constraints	Wall cap	Closed (INF)	UNKNOWN	Aggregate solver wall time
CP-SAT, window-bounds	3-AP + window + symmetry	300 s	55 / 100	45 / 100	bounded by ~8.3 wall-h
HiGHS, window-bounds	3-AP + window + endpoint	3,600 s	**0 / 45**	45 / 45	45.0 wall-h

Across 162,003 solver-seconds and 3,181,316 explored MIP nodes, HiGHS did not close a single chunk. No `FEASIBLE` row appeared. The recorded HiGHS dual bound stayed at its uninformative default value in every task, which is itself evidence that the LP-relaxation route was not discovering a useful certificate for these instances.

We then re-attacked all 45 T1 chunks under an extended 8-hour HiGHS wall, with LP progress logging enabled. The result is mixed: 25/45 chunks closed `INFEASIBLE`, while 20/45 returned `UNKNOWN` at the full cap with dual bound still pinned at 0.0. We call this 20-chunk LP-paradigm-resistant subset **T1b**. The audit consumed 901,073 solver-seconds and 25,196,448 MIP nodes.

To test whether T1b is invariant under solver architecture more broadly, we re-attacked it with a CDCL/SAT solver (CaDiCaL via PySAT [BFFH20, IMMS18], single-threaded, 4-hour wall, encoding restricted to 3-AP triples + cardinality + chunk pins; no window-cardinality clauses). CDCL closed 18/20 chunks `UNSAT`. We call the surviving 2-chunk residual $\mathbf{T1c} = \{40959, 48895\}$. A T1c diagnostic at extended wall (12 h pure CDCL) and with totalizer-encoded window constraints for lengths `{31, 100, 199}` (4 h) also returned `UNKNOWN` on both chunks. At that point in the campaign, T1c appeared resistant to every tested paradigm; §4.4 reports the re-attack that overturned this.

A proof-producing rerun of the 18 CDCL-UNSAT chunks initially emitted DRAT proofs for 17 chunks; the remaining chunk, 32735, required a larger-memory proof rerun and then also emitted a DRAT proof. Independent `drat-trim` verification confirmed all 18/18 emitted proofs. The final two certificates were the slowest: 63231 verified in 49,758.11 seconds and 32735 verified in 80,960.68 seconds under the final 24-hour verifier pass [HHB13, CFHH+17].

The picture after the CaDiCaL pass: **LP-paradigm methods (CP-SAT constraint propagation and HiGHS LP-relaxation MIP) fail uniformly on T1b**, and the CDCL clause-learning paradigm closes 18/20 of those chunks, all of which are independently verified. The apparent paradigm-invariant residual was T1c, of size 2.

4.4. **Kissat closes T1c: the boundary is implementation, not paradigm.** The CaDiCaL result left open whether T1c marks the edge of the CDCL paradigm or only of one implementation. We therefore re-attacked both chunks with kissat 4.0.4 [BFFH20], single-threaded, on the same pure 3-AP/cardinality encoding (no window clauses), with DRAT logging enabled. Both chunks were refuted, and both refutations are certified end-to-end: against sha256-pinned formula files, chunk 40959 solves in 23,191 s with independent `drat-trim` verification in 41,127 s, and chunk 48895 solves in 10,083 s with verification in 16,153 s. (An initial kissat run on the same encoding had already refuted both, in 14,204.9 s and 17,621.7 s, emitting proofs of 17.4 and 14.3 GB; the certified rerun re-solved and re-verified against the same pinned formula file per chunk to establish clean certificate provenance.) Every solve sits well inside the 12-hour budget under which the CaDiCaL attempts failed. One caveat on attributing this gap: the CaDiCaL attacks ran through PySAT's bundled build with default configuration, so part of the separation may reflect build vintage or configuration defaults rather than core search differences; the released instances make it straightforward to re-measure against any solver build. (A windowed kissat variant exceeded the 16 GB encode-time memory budget on both chunks and produced no solver result.)

To measure whether the kissat/CaDiCaL gap is specific to T1c or generic across the broad residual, we ran a kissat survey on a fresh uniform random 100-chunk sample of the 6,071 T2 `UNKNOWN`s (2-hour cap, no proof logging): 99/100 closed `UNSAT`, with median solve time 121.7 s and maximum 2,408.8 s; the single survivor (chunk 98301) hit the cap. The whole survey consumed 34,819.8 solver-seconds — less than ten CPU-hours for 99 closures of subproblems that all survived the campaign's 60-second CP-SAT broad pass with window bounds. Together with the T1c closures, this collapses the hard pocket as a paradigm phenomenon. What remains true, and is now cleanly isolated: (i) the LP-flatness failure of CP-SAT propagation and MIP on this family is real and uniform; (ii) within CDCL, implementation matters decisively at the hard end — the two chunks that defeat CaDiCaL at 12 h fall to kissat in under 5 h; and (iii) the tiers T1a/T1b/T1c retain their value as a graded, ground-truth-verifiable ladder that separates solver technologies at three distinct heights.

4.5. **Levers tested and their outcomes.** For completeness, we record every search-tuning lever we tested on the hard bucket $[61575, 66575)$ or the 300s-resistant subset. The CP-SAT/MIP-side tuning levers did not remove the hard pocket; the one qualitative exception is the CDCL paradigm switch, which closes most of T1b and therefore narrows, rather than merely tunes, the residual.

Lever	Mechanism	Result
Window-cardinality (OEIS A003002)	Add $\sum x \leq r_3(L)$ for each window	UNK rate on full 10k batch: 30.33% → 1.65%. Strong but plateaus.
Split-vars reorder (hot pins first)	Permute the depth-24 split prefix	13.60% → 12.96% on the worst bucket. Noise.
Wall-cap extension	60s → 300s on the broad pass	Saturating curve, see §4.2.
Targeted pair-AND Tseitin propagators	Explicit `pair[a,c] = x[a]∧x[c]` BoolVars on midpoints in $[67, 91]$	Control 67.84s, treatment 71.75s on a sampled residual. No effect.
Recursive deepening	Refinement at depths 40, 48, 56	All 500 stratified-sample base chunks close, but with a non-trivial tail; the final six level-3 residuals needed up to 599.74s at the 600s cap.
HiGHS MIP with LP/cut machinery	Replace CP-SAT entirely	0/45 closed at 1 hour; 25/45 closed in the full 8-hour audit, while 20/45 retained dual bound 0.0.
Pure CDCL/SAT	Encode T1b as CNF with 3-AP clauses + cardinality + pins, no windows	18/20 HiGHS-flat chunks closed `UNSAT` in 4 hours; 2/20 remained `UNKNOWN`; no `SAT` rows. This breaks the strong solver-invariance framing.
Kissat re-attack (§4.4)	Same pure CNF encoding, kissat 4.0.4	Both T1c chunks refuted in under 5 h; 99/100 of a fresh T2 sample closed within 2 h. Eliminates the residual as a paradigm boundary.

The pair-AND result is worth a closer note. The added constraints are mathematically redundant with the existing 3-AP triple inequalities — they encode the same forbidden configurations, just with an explicit Tseitin variable per pair. The hope was that this would give CP-SAT more "propagation hooks" in the structural pocket. The negative result is consistent with the broader picture: the pocket's hardness is not an encoding issue, it is a search-space issue.

4.6. **A refuted conjecture about the pocket.** During the campaign we maintained a working conjecture about T1c, which the kissat re-attack of §4.4 refuted. We record both the conjecture and its refutation, because the refutation is the more informative result.

The conjecture held that each T1c subproblem corresponds to a depth-24 fixed assignment in which `(i)` the LP relaxation upper bound on $\sum x_i$ is at most one above the decision threshold $K = 44$, leaving no room for LP-based cuts to prove infeasibility, and `(ii)` the integer infeasibility certificate is too combinatorially diffuse to fit in the working memory of current CDCL clause-learning solvers under practical wall caps. Part (i) survives: the LP-flatness signature is measured fact, uniform across T1b and visible even on the monolithic instance (§3.1). Part (ii) is false as stated: kissat 4.0.4 finds refutations for both chunks well within the failed CaDiCaL budget, with multi-gigabyte DRAT proofs that `drat-trim` independently verified. The certificates are large but not beyond reach; what CaDiCaL-at-12h lacked was not memory for the certificate but the search heuristics to find it.

The corrected picture: closing this family's hard instances requires either LP-side inference that lifts a dual bound off zero — which no tested MIP/CP configuration achieved — or CDCL implementations at the current competition frontier. The gap between CDCL implementations on these instances (a factor of at least 2.4× in wall time, and in practice closed-vs-open) is itself a benchmark finding: T1c separates solvers *within* a paradigm, which is rarer and arguably more useful than separating paradigms.

4.7. **What this means for $r_3(212)$.** The combined CP-SAT, HiGHS, CaDiCaL, and kissat evidence — 0 `FEASIBLE/SAT` rows across millions of subproblems, every audited hard-pocket chunk closed `UNSAT`, and 99/100 of a fresh broad-residual sample closed within 2 hours — strongly supports the OEIS-conjectured value $r_3(212) = 43$, and the kissat results substantially change the outlook on proving it. The lower bound $r_3(212) \geq 43$ from the verified witness in §2 is unaffected; the trivial upper bound $r_3(212) \leq 44$ follows from monotonicity and OEIS $r_3(211) = 43$, so the gap between our certified bounds is exactly one.

What blocks the upper bound is now engineering scale, not an unsolved search obstruction: closing it requires certifying the full 12,582,912-chunk depth-24 sweep (T3), of which this campaign processed the 100,000-chunk expansion plus samples. At the T2 pilot's observed rate (99/100 within 2 h/chunk, single-core), a full kissat sweep of the remaining lattice is a bounded, parallelizable computation — large, but of the same character as the Pythagorean-triples and Schur-five campaigns [HKM16, Heu18], with the witness-split generator providing the cube decomposition. We release the benchmark instances and generator in §5 so that this sweep, or a sharper per-chunk method, can be attempted directly.

5. Open problem and benchmark release

The campaign establishes strong empirical support for $r_3(212) = 43$ (§3.4, §4.4) and shows that the remaining obstacle to a full upper-bound certificate is scale, not an unsolved search problem: kissat closes every audited hard instance and 99% of a fresh residual sample (§4.4). This section states the residual problem cleanly and releases the instances as a tiered benchmark — both as the concrete path to finishing R3-212-UB and as a solver-grading ladder that separates LP-paradigm methods from CDCL, and CDCL implementations from one another.

5.1. Statement of the open problem.

> **Problem (R3-212-UB).** Let $A \subseteq [1, 212]$ with `|A| = 44`. Determine whether there exists A containing no nontrivial 3-term arithmetic progression. The verified 43-element witness in `results/N212_K43_witness.json` (§2.1) and the value $r_3(211) = 43$ from OEIS A003002 jointly imply that any such A must contain both endpoints 1 and 212. Equivalently, either exhibit one such A or certify the infeasibility of the decision problem $\sum_i x_i = 44$ over the 3-AP constraint family on $[1, 212]$ with $x_1 = x_{\{}212\} = 1$.

A solution to R3-212-UB resolves the unit gap $r_3(212) \in \{43, 44\}$ in either direction. Our multi-million-subproblem CP-SAT campaign, the 45-instance HiGHS attack, and the CaDiCaL/kissat re-attacks returned 0 `FEASIBLE/SAT` rows, which is evidence for the infeasibility branch but not a proof. The benchmark instances released below pinpoint the subproblems on which solver technologies diverge.

5.2. **Benchmark instance tiers.** We release three tiered benchmark sets. Each JSONL row records the chunk ID, fixed assignments, witness-pin prefix, solver status, and timing data; the common constraint family (3-AP triples, endpoint forcing, and OEIS window bounds) is reconstructed by the repository scripts. The tiers form a ladder from the smallest solver-resistant pocket to the full upper-bound proof.

Tier	Artifact	Size	Resistance level	Recommended target
T1a	T1a JSONL	25 chunks	closed by HiGHS at 8h (dual = `inf`)	reference / regression test
T1b \ T1c	T1b-minus-T1c JSONL	18 chunks	LP-paradigm-resistant; closed by CDCL; all 18/18 emitted DRAT proofs verified by `drat-trim`	certified CDCL benchmark
T1c	T1c JSONL	2 chunks	resist CP-SAT, HiGHS LP-MIP, CaDiCaL @ 12h; refuted by kissat 4.0.4, DRAT-certified	CDCL-implementation calibration point
T2	T2 JSONL	6,071 chunks	survived 60-s CP-SAT broad pass with window bounds	full closure of the expansion residual
T3	deterministic generator	12,582,912 chunks	unprocessed remainder of the witness-split lattice	full upper-bound proof

The concrete filenames are `results/N212_K44_t1a25.jsonl`, `results/N212_K44_t1b_minus_t1c.jsonl`, `results/N212_K44_t1c2.jsonl`, and `results/N212_K44_window100k_unknowns.jsonl`.

Tier T1c is the benchmark's calibration point for CDCL implementations: the two chunks defeat CaDiCaL at a 12-hour wall and fall to kissat 4.0.4 well within that budget, with end-to-end certified DRAT proofs (§4.4). A solver or configuration that closes T1c quickly is at the current hard frontier of this family; one that certifies it more cheaply than the 41,127-second worst-case verification reported here improves on the state of the art.

Tier T2 is the canonical "close the campaign at the broad layer" target: closing all 6,071 chunks of the 100k expansion batch is necessary, though not sufficient, for a full upper-bound proof. The kissat pilot (99/100 within 2 h/chunk at median 121.7 s, §4.4) extrapolates to roughly 500 single-core CPU-hours of solving for the full tier, plus proof-logging and verification overhead and whatever the $\approx 1\%$ tail beyond the 2-hour horizon costs; chunk 98301 — the one pilot survivor — is the natural first target for anyone probing that tail.

Tier T3 is the full upper-bound certificate: closing the entire depth-24 AP-pruned sweep. The instance generator (§2.2) emits the required chunks deterministically from the witness file and the OEIS b-file; the storage cost of the full sweep is dominated by output rather than input.

5.3. **Minimum-viable proof requirements.** A formal proof of $r_3(212) \leq 43$ from the released benchmark requires:

1. **Closure of tier T3.** Every chunk in the depth-24 AP-pruned sweep must return `INFEASIBLE` under a verified solver, or one chunk must return a `FEASIBLE` 44-element witness.
2. **Solver verification.** We recommend that any solver used for closure either produce machine-checkable proof certificates (DRAT, LRAT, or equivalent), or be independently reproduced under a second solver paradigm. This is a verification target rather than a condition already met by every row of the present campaign; §6.2 records the remaining certificate gaps.
3. **Constraint-set verification.** The 11,130 3-AP triple inequalities, the 22,154 window-cardinality inequalities, and the endpoint forcing must be checked against the formal definition of $r_3(N)$. The verifier `r3_verify.py` performs this check on any candidate witness; an analogous check for the constraint generation is included in the repository.

The minimum-viable result short of a full proof is now the certified closure of tier T2: verified proofs for all 6,071 broad-residual chunks. The kissat pilot (§4.4) prices the solving at roughly 500 single-core CPU-hours (extrapolating the pilot's mean closed-chunk time to all 6,071 chunks), with proof emission, verification, and the $\approx 1\%$ hard tail as the real cost drivers — a bounded campaign rather than an open research question. An LP-paradigm result that lifts a dual bound off 0.0 on any T1b chunk would be a separate, qualitatively new contribution, since no MIP/CP configuration we tested achieved it (§3.1, §4.3).

5.4. **Approaches we could not test.** The following techniques are plausibly stronger than CP-SAT and HiGHS on this pocket but were outside the scope of our campaign. We flag them as natural follow-on directions:

- **Fourier-analytic upper bounds.** Behrend-style constructions achieve $r_3(N) = N \cdot exp(-c \cdot sqrt(logN))$ lower bounds; a matching Fourier-analytic upper bound for small N would give a certificate independent of combinatorial search. The Bloom–Sisask framework for $r_3(N) = O(N/(logN)^{\{}1+c\})$ is asymptotic but the underlying density-increment machinery may yield finite-N bounds tighter than the OEIS window-cardinality family.
- **Multi-window partition bounds.** OEIS A003002 is used here as a single-window family `sum_{[a, a+L)} x_i` $\leq$ `r_3(L)`. A partition-cardinality bound across multiple disjoint windows of varying length might cut the LP relaxation more aggressively than any single-window family.
- **SAT with proof logging at scale.** A pure CNF encoding closed 18/20 T1b chunks under CaDiCaL (all DRAT-verified) and, per §4.4, the remaining two under kissat 4.0.4 (also DRAT-verified). The research target has therefore moved from single instances to the certified sweep: proof-producing kissat over all of T2 and ultimately T3, with verification and archival at Pythagorean-triples scale [HKM16]. Open per-instance questions remain at the margins — e.g. chunk 98301, the one pilot survivor, and whether alternative cardinality encodings shrink the multi-GB certificates of the hardest chunks.
- **Lean/formal-proof-search benchmark.** The repository now includes a compact Lean 4 / Mathlib 4 formalization [dMU21, The20] under `lean/`: shared definitions (`R3Base.lean`), the verified 43-witness statement (`R3_212_Witness.lean`), and two AlphaProof-style T1c targets (`R3_T1c_40959.lean`, `R3_T1c_48895.lean`) with the expected answer `false`. These files are intended as a starting point for an A003002 / $r_3(N)$ entry in formal-conjecture repositories [Goo26]. Recent AlphaProof Nexus-style workflows have resolved Erdős problems of comparable complexity at low per-problem cost when a Lean target exists [TKS+26], so T1c is well-shaped for that paradigm.
- **Custom branch-and-bound.** A solver that exploits problem- specific symmetries and dominance relations — for instance, reflection symmetry after endpoint forcing, or domination

of one window-bound by another at specific fixed-pin configurations — could prune branches that neither CP-SAT nor HiGHS recognize as redundant.
- **Symmetric Difference Encoding / Lasserre hierarchy at low degree.** A degree-4 or degree-6 SDP relaxation of the 3-AP-free constraint might separate the surviving T1 instances from feasibility where the LP relaxation cannot.

We do not advocate for any single direction. The benchmark release is intended to let specialists pick the technique closest to their toolkit.

5.5. **Reproducibility and release.** The campaign code, configuration, and result logs are released at the repository accompanying this preprint. Specifically:

- All Python sources implementing the architecture of §2 are present with module-level documentation.
- The SLURM scripts (`submit_*.sbatch`) and the driver `unity_handoff.sh` reproduce the full campaign on any SLURM-compatible cluster with OR-Tools and `highspy` installed.
- The verified 43-witness, the OEIS A003002 b-file used for window bounds, the broad-pass result logs, the recap residuals, the HiGHS attack logs, the T1a/T1b/T1c benchmark JSONLs, verified DRAT certificates, LRAT artifacts where emitted with `drat-trim -L`, the Lean T1c proof-search targets, environment/version captures for the main solver stack, and the scripts needed to generate T3 are all included.
- Existing single-instance entry points (`r3_split_cpsat.py` and `r3_highs_attack.py`, and `r3_sat_attack.py`) can rerun any row of T1c, T1b, or T2 by chunk ID.

The large solver artifacts are archived on Zenodo at https://doi.org/10.5281/zenodo.20463334. This archive contains the CNF/DRAT proof artifacts for the 18 CDCL-closed T1b T1c chunks, the available LRAT outputs, verification summaries, SLURM logs, solver outputs, SHA256 manifests, and reconstruction instructions for split archives [Erg26].

We welcome correspondence on partial results: any solver run that closes a strict subset of T1 or T2 is a meaningful incremental contribution, even if the full upper bound remains open.

6. Discussion

We close with three observations: what the architecture transfers to, where it breaks down, and what the campaign suggests about the broader proof-search problem class for r_3 upper bounds.

6.1. **Reusability for adjacent N.** The five-component architecture of §2 is parametric in N and K and depends on two external inputs: a verified `(K - 1)`-element lower-bound witness for $r_3(N)$ and the OEIS A003002 b-file prefix for window-cardinality pruning at lengths `L ≤ N - 1`. Both inputs are standard finite data at the current frontier: exact values through $N = 211$ are tabulated by Cariboni/OEIS A003002, and any new run also requires an explicit verified lower-bound witness at the target size. We expect the architecture to apply directly to `N ∈ \{213, ..., 220\}` once such witnesses are supplied, with three caveats:

1. The depth of the broad split ($d = 24$ for $N = 212$) is empirical and likely needs to grow with N. The number of feasible-after-pruning chunks at depth d scales roughly geometrically with the size of the witness, so the broad layer for larger N will emit more chunks, and per-chunk cost will also rise as the number of variables and 3-AP triples grows.
2. The $r_3(N)$ value drops slowly as N grows, so the equality $\sum x_i = K$ becomes harder to refute by simple cardinality arguments. We expect the `UNK` rate at the 60-s wall cap to grow with N.

3. The endpoint-forcing argument generalizes: if $r_3(N-1) = K-1$, then any K-element 3-AP-free subset of $[1, N]$ contains both endpoints. This is the only structural input from prior work that the architecture exploits; everything else is N-uniform.

Within the OEIS A003002 frontier, the architecture is therefore a drop-in tool for any incremental $r_3(N')$ upper-bound attempt, with the same caveat the present campaign documents: the hard pocket of §4 will likely have an analogue at larger N, possibly more severe — and whether that analogue again falls to the strongest available CDCL implementation, as it did at $N = 212$, is an open empirical question.

6.2. **Limitations.** The campaign's main limitations are certificate coverage and scale: most closures are solver-attested rather than third-party-verified, and the processed chunk ranges cover a fraction of the full depth-24 sweep. Section 5 prices the remaining path; here we record the narrower limitations of the present implementation:

- **Machine-checkable certificates cover every CDCL-resolved hard-pocket chunk, but not the full campaign.** Follow-up proof-producing runs emitted DRAT proofs for all 18 CaDiCaL-resolved T1b \ T1c chunks, and independent `drat-trim` verification confirmed all 18/18, and the two kissat T1c proofs are likewise verified end-to-end against sha256-pinned formulas (§4.4). The 99 kissat T2-survey closures ran without proof logging by design; the 25 HiGHS-closed T1a chunks and the ~93,929 broad-pass CP-SAT INFEASIBLE returns remain solver-attested rather than third-party-verified. Closing the CP-SAT/MIP side of this gap would require either a SAT re-encoding of each chunk or a verified LP-duality-style certificate format we are not aware of in the open MIP ecosystem.
- **Compute coverage is incomplete.** The campaign processed 100,000 of the 12,582,912 AP-pruned depth-24 chunks at the broad layer, plus refinements and kissat surveys on samples. The kissat results arrived at the end of the campaign; a full certified sweep under a kissat-first strategy (priced in §5.3 for T2) is the natural successor project rather than a rerun of this one.
- **Witness dependence is structural, not adversarial.** The broad split is anchored to one specific 43-witness. A different witness would induce a different chunk-ID space and possibly a different hard pocket. We did not test whether the hard pocket is witness-invariant; if it is not, alternative witnesses might bypass the T1c residual of §4.3.
- **The Lean benchmark is not yet upstreamed.** We did not find a public A003002 / $r_3(N)$ entry in `google-deepmind/formal-conjectures` as of the campaign date. The repository includes a Lean 4 formalization of the witness and T1c targets, but upstreaming and independent typechecking in that ecosystem remain follow-on work.

The third limitation is the most interesting follow-on: a witness- ensemble version of the broad split, in which the chunk space is the disjoint union of split-prefixes from multiple distinct 43-witnesses, would either reveal an easier alternative covering of the search space or confirm that the hard pocket is intrinsic to the constraint family rather than to one witness.

6.3. **What the campaign suggests about r_3 upper-bound search.** The dominant finding is now more precise than the original CP-SAT/HiGHS comparison. LP-paradigm methods — CP-SAT-style propagation on the bounded model and HiGHS LP-MIP — fail uniformly on the 20 T1b chunks with flat dual bounds, while CDCL closes 18/20 of them. The residual T1c set has size 2; it defeated CaDiCaL but not kissat, so the sharpest surviving separations are LP-paradigm-vs-CDCL and CaDiCaL-vs-kissat, not paradigm-vs-nothing.

Two narrower suggestions follow from the lever inventory of §4.5. First, the window-cardinality family from OEIS A003002 is doing nearly all of the work that generic constraint propagation can do; the ~28-percentage-point reduction it produces is not matched by any other intervention at any cost. Second, the levers that *should* have moved the residual under standard MIP/CP intuition — variable reordering, pair-AND Tseitin propagators, walltime extension to the plateau — moved it by less than two percentage points each. The residual is not lying in wait for a smarter generic search.

The exception is the CDCL family (§4.3–§4.4), which closes everything: 18/20 of T1b under CaDiCaL and the remainder under kissat. We treat the paradigm switch as a qualitative change of proof system, and the CaDiCaL-to-kissat gap as evidence that, at the hard end of this family, solver engineering currently matters as much as proof system.

Combined, these suggest that the most productive next step is not another CP-SAT or MIP variant but the certified kissat sweep of T2 and T3 priced in §5.3 — with two research-grade side questions left open: (i) can any LP-paradigm method lift a dual bound off 0.0 on a T1b chunk, and (ii) what feature of kissat 4.0.4's search accounts for the CaDiCaL-vs-kissat separation on T1c? The benchmark release of §5 is built to support exactly this kind of follow-on.

The campaign's working conjecture (§4.6) — that T1c's integer infeasibility certificate was beyond the reach of current CDCL under practical wall caps — was refuted by the solvers themselves. That outcome is worth generalizing: on structured additive-combinatorics instances, "paradigm-resistant" claims have a short shelf life, and benchmark releases that archive the resistant instances — rather than merely reporting their existence — are what allow such claims to be tested, and overturned, quickly.

Appendix A. Compute accounting

The retained logs provide measured solver wall time for the main diagnostics. Since CP-SAT tasks used 8 workers, the worker-hour estimates below multiply recorded solver-wall seconds by 8; for HiGHS, the analogous estimate is 8 threads per one-hour task. These are retained-log estimates, not exact SLURM accounting totals.

Layer	Worker-hours	Notes
Retained broad logs	~2,276	includes the 10k no-window run, the 10k window run, the 100k window run, and the $[575, 675)$ A/B
Sample-100 refinement	~375	depth-40 plus one depth-48 tail
Sample-500 refinement	~1,978	depth-40, depth-48, depth-56, and depth-64 cleanup
Recap and worst-bucket A/Bs	~598	reorder, 300s walltime, 120s recap, 300s recap
HiGHS attack on 45 survivors	~360	1-h wall, 8 threads per task
Full T1 HiGHS long-wall audit	~2,002	45 T1 chunks, 8-h cap, job 58782313; estimated from retained `seconds` fields
T1b CDCL first run	~19.8	20 single-core tasks, retained JSON row time 71,216.85 seconds
T1b proof-producing rerun + T1c diagnostic	~120	proof emission + 4-cell T1c grid
drat-trim verification	~61	jobs 58952708, 59058393, and 59383874; all 18 CDCL-resolved `T1b \ T1c` chunks VERIFIED
Total retained logs	~7,850 worker-hours	excludes interactive debugging and logs not retained locally

The campaign fits within a single academic SLURM allocation on a CPU partition. SLURM job identifiers are retained in the table and in the released logs so that every worker-hour estimate can be traced to a specific recorded run.

Acknowledgments

This work was performed on the Unity High Performance Computing (HPC) platform, a collaborative, multi-institutional resource supported by the Massachusetts Green High Performance Computing Center (MGHPCC) and its member institutions. We acknowledge OEIS contributor Cariboni for the A003002 b-file at the current frontier $N \leq 211$.

Use of generative AI

In the preparation of this work, the author used Anthropic Claude (Claude Sonnet 4.5 and Claude Opus 4.6, accessed via the Claude Code agent interface) and OpenAI Codex (GPT-5, accessed via the Codex agent interface) for drafting and revision assistance on the manuscript, repository scaffolding, and computational orchestration. The author reviewed and verified every claim and every line of code before submission, and takes full responsibility for the content. These tools did not generate mathematical claims, did not direct the campaign's experimental design, and do not appear as authors.

References

[Beh46] F. A. Behrend, *On sets of integers which contain no three terms in arithmetical progression*, Proceedings of the National Academy of Sciences **32** (1946), no. 12, 331–332.

[BFFH20] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximilian Heisinger, *CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT competition 2020*, Proceedings of SAT Competition 2020: Solver and Benchmark Descriptions, Department of Computer Science Report Series B, University of Helsinki, 2020, pp. 51–53.

[BHMN22] Joshua Brakensiek, Marijn J. H. Heule, John Mackey, and David Narváez, *The resolution of Keller's conjecture*, Journal of Automated Reasoning **66** (2022), 277–300.

[BS20] Thomas F. Bloom and Olof Sisask, *Breaking the logarithmic barrier in Roth's theorem on arithmetic progressions*, arXiv preprint (2020), arXiv:2007.03528.

[Car24] Lorenzo Cariboni, *b-file for OEIS A003002 up to* $n = 211$, OEIS A003002 b-file, 2024.

[CFHH+17] Luís Cruz-Filipe, Marijn J. H. Heule, Warren A. Hunt, Jr., Matt Kaufmann, and Peter Schneider-Kamp, *Efficient certified RAT verification*, CADE-26 (2017), 220–236.

[dMU21] Leonardo de Moura and Sebastian Ullrich, *The Lean 4 theorem prover and programming language*, CADE-28, 2021, pp. 625–635.

[Dyb12] Janusz Dybizbański, *Sequences containing no 3-term arithmetic progressions*, Electronic Journal of Combinatorics **19** (2012), no. 2, #P15.

[Erg26] Mehmet Ergezer, *Artifacts for the* $r_3(212)$ *witness-split computational campaign*, Zenodo dataset, 2026, https://doi.org/10.5281/zenodo.20463334.

[GHG+21] Ambros Gleixner, Gregor Hendel, Gerald Gamrath, Tobias Achterberg, Michael Bastubbe, Timo Berthold, Philipp Christophel, Kati Jarck, Thorsten Koch, Jeff Linderoth, Marco Lübbecke, Hans D. Mittelmann, Derya Ozyurt, Ted K. Ralphs, Domenico Salvagnin, and Yuji Shinano, *MIPLIB 2017: Data-driven compilation of the 6th mixed-integer programming library*, Mathematical Programming Computation **13** (2021), 443–490.

[GMN22] Stephan Gocht, Ciaran McCreesh, and Jakob Nordström, *An auditable constraint programming solver*, 28th International Conference on Principles and Practice of Constraint Programming (CP 2022), LIPIcs, vol. 235, Schloss Dagstuhl, 2022, pp. 25:1–25:18.

[Goo26] Google DeepMind, *formal-conjectures: A lean library of open mathematical conjectures*, 2026, Accessed 2026-05-25.

[GS04] Carla P. Gomes and Meinolf Sellmann, *Streamlined constraint reasoning*, Principles and Practice of Constraint Programming – CP 2004, Lecture Notes in Computer Science, vol. 3258, Springer, 2004, pp. 274–289.

[GW99] Ian P. Gent and Toby Walsh, *CSPLib: A benchmark library for constraints*, Principles and Practice of Constraint Programming – CP 1999, Lecture Notes in Computer Science, vol. 1713, Springer, 1999, pp. 480–481.

[Heu18] Marijn J. H. Heule, *Schur number five*, Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18), 2018, pp. 6598–6606.

[HH18] Q. Huangfu and J. A. J. Hall, *Parallelizing the dual revised simplex method*, Mathematical Programming Computation **10** (2018), no. 1, 119–142, HiGHS solver; https://highs.dev.

[HHB13] Marijn J. H. Heule, Warren A. Hunt, Jr., and Armin Biere, *Trimming while checking clausal proofs*, FMCAD 2013, IEEE, 2013, pp. 181–188.

[HKM16] Marijn J. H. Heule, Oliver Kullmann, and Victor W. Marek, *Solving and verifying the boolean Pythagorean triples problem via cube-and-conquer*, Theory and Applications of Satisfiability Testing – SAT 2016, Lecture Notes in Computer Science, vol. 9710, Springer, 2016, pp. 228–245.

[HKWB12] Marijn J. H. Heule, Oliver Kullmann, Siert Wieringa, and Armin Biere, *Cube and conquer: Guiding CDCL SAT solvers by lookaheads*, Hardware and Software: Verification and Testing (HVC 2011), Lecture Notes in Computer Science, vol. 7261, Springer, 2012, pp. 50–65.

[IMMS18] Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva, *PySAT: A Python toolkit for prototyping with SAT oracles*, SAT 2018, 2018, pp. 428–437.

[KM23] Zander Kelley and Raghu Meka, *Strong bounds for 3-progressions*, arXiv preprint (2023), arXiv:2302.05537.

[OEI24] OEIS Foundation Inc., *The on-line encyclopedia of integer sequences, sequence A003002: Salem–spencer numbers*, 2024, b-file extended by L. Cariboni; accessed 2026.

[PF24] Laurent Perron and Vincent Furnon, *OR-Tools, version 9.x*, 2024.

[Rot53] K. F. Roth, *On certain sets of integers*, Journal of the London Mathematical Society **28** (1953), 104–109.

[SS42] R. Salem and D. C. Spencer, *On sets of integers which contain no three terms in arithmetical progression*, Proceedings of the National Academy of Sciences **28** (1942), no. 12, 561–563.

[The20] The Mathlib Community, *The Lean mathematical library*, CPP 2020, 2020, pp. 367–381.

[TKS+26] George Tsoukalas, Anton Kovsharov, Sergey Shirobokov, Anja Surina, Moritz Firsching, Gergely Bérczi, Francisco J. R. Ruiz, Arun Suggala, Adam Zsolt Wagner, Eric Wieser, Lei Yu, Aja Huang, Miklós Z. Horváth, Andrew Ferrauiolo, Henryk Michalewski, Codrut Grosu, Thomas Hubert, Matej Balog, Pushmeet Kohli, and Swarat Chaudhuri, *Advancing mathematics research with AI-driven formal proof search*, arXiv preprint (2026), arXiv:2605.22763.

Wentworth Institute of Technology, Boston, MA, USA

Email address: ergezerm@wit.edu